

Relatório Técnico — Desafio DevOps

Automação de CI/CD, Containerização Multi-Stage e Orquestração Multi-Ambiente

Luan Chaves — [RM562814](#)

Pietro — [RM564024](#)

Matheus Nicacio — [RM564257](#)

Nicolas Guilherme — [RM561466](#)

1. Introdução e Contexto do Projeto

O projeto **Cidades ESG Inteligentes** é uma solução corporativa desenvolvida em **Java 21 LTS** e **Spring Boot 3.3.3** integrada ao banco NoSQL **MongoDB 7.0**. A aplicação foi concebida para atender à gestão pública e à iniciativa privada no monitoramento de telemetria ambiental (qualidade do ar, ruído urbano, consumo hídrico e energético), cálculo de pegada de carbono setorial e governança de projetos municipais de sustentabilidade.

O objetivo deste trabalho é consolidar práticas de **DevOps**: build, testes, cobertura, empacotamento em imagem Docker e deploy automatizado em ambientes acadêmicos isolados de **Staging** e **Produção**.

2. Pipeline de CI/CD (GitHub Actions)

A esteira de integração e entrega contínua foi implementada através do **GitHub Actions** no arquivo `.github/workflows/ci-cd.yml`. O fluxo automatizado é acionado em eventos de `push` e `pull_request` nas branches estratégicas (`main` e `develop`).

Detalhamento das Etapas do Pipeline:

- Build & Testes Automatizados:** Provisiona o ambiente com JDK 21 (Eclipse Temurin), restaura o cache Maven, compila o código e executa a suíte de testes unitários e de integração (JUnit 5 + Mockito). Gera relatório de cobertura JaCoCo.
- Docker Multi-Stage:** Constrói a imagem e interrompe a esteira se o Dockerfile não for funcional.
- Deploy Automatizado em Staging:** Sobe o ambiente de homologação via Docker Compose Staging na porta 8081 e executa testes de fumaça (Smoke Tests).
- Deploy Automatizado em Produção Acadêmica:** Sobe o Compose de produção na porta 8080 e valida health e dashboard. Os ambientes são efêmeros para dispensar infraestrutura paga.

```
# Trecho da Esteira GitHub Actions (.github/workflows/ci-cd.yml) name: CI/CD Pipeline - Cidades ESG
Inteligentes on: push: branches: [ main, develop ] pull_request: branches: [ main, develop ] jobs: build-and-
test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-java@v4 with:
{ distribution: 'temurin', java-version: '21', cache: 'maven' } - run: bash ./mvnw clean verify -B
```

3. Containerização e Otimização com Docker

Para garantir eficiência, segurança e redução do consumo de banda e armazenamento, adotou-se o padrão **Multi-Stage Build**. O build é segmentado em dois estágios distintos:

- **Stage 1 (Builder):** Utiliza `maven:3.9.8-eclipse-temurin-21-alpine` para resolução de dependências e empacotamento do arquivo `.jar`.
- **Stage 2 (Runner):** Utiliza a imagem mínima `eclipse-temurin:21-jre-alpine` contendo apenas o runtime JRE 21, reduzindo a imagem final para ~180MB.

Boas Práticas de Segurança e Operação Adotadas:

- **Execução Non-Root:** Criação do usuário `appuser:appgroup` para evitar que a aplicação execute com privilégios de `root`.
- **Tratamento de Sinais POSIX (dumb-init):** Garante o encerramento seguro (Graceful Shutdown) sem deixar processos zumbis.
- **Otimização de JVM em Containers:** Parâmetros `-XX:+UseContainerSupport` e `-XX:MaxRAMPercentage=75.0`.
- **Healthcheck Nativo:** Integração direta com `/actuator/health` via probe periódica.

4. Orquestração de Ambientes: Staging e Produção

Ambiente	Orquestrador	Porta API	Banco de Dados	Estratégia de Deploy
Local	Docker Compose	8080	MongoDB Local (27017) + Mongo Express (8081)	<code>docker compose up -d --build</code>
Staging	Docker Compose Staging	8081	MongoDB Staging (27018)	Automático via CI/CD (branch develop/main)
Produção	Docker Compose Produção	8080	MongoDB Produção com volume isolado	Automático via CI/CD (branch main)

5. Evidências de Execução e Funcionamento

✓ 5.1 Testes Automatizados Aprovados (Maven / JUnit)

```
[INFO] Running br.com.fiap.cidadesesg.SensorServiceTest [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 - SUCCESS [INFO] Running br.com.fiap.cidadesesg.EmissionServiceTest [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0 - SUCCESS [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0 [INFO] Analyzed bundle 'cidades-esg-inteligentes' with 20 classes [INFO] BUILD SUCCESS
```

✓ 5.2 Status dos Containers em Execução (Docker Compose)

```
NAME IMAGE STATUS PORTS cidades-esg-api-staging fiap-devops/cidades-esg-api:staging Up (healthy) 0.0.0.0:8081->8081/tcp mongodb-esg-staging mongo:7.0 Up (healthy) 0.0.0.0:27018->27017/tcp cidades-esg-api-prod fiap-devops/cidades-esg-api:prod Up (healthy) 0.0.0.0:8080->8080/tcp
```

✓ 5.3 Resposta do Endpoint Actuator Health (UP)

```
{ "status": "UP", "components": { "diskSpace": { "status": "UP" }, "mongo": { "status": "UP" }, "readinessState": { "status": "UP" } } }
```

6. Desafios Encontrados e Soluções Implementadas

Desafio 1: Gerenciamento de Memória da JVM no Container Alpine

Problema: Aplicações Java legadas frequentemente ignoravam os limites de memória estipulados pelo cgroups do Docker, resultando em encerramentos abruptos por Out-Of-Memory (OOMKilled).

Solução: Configuração explícita das flags do OpenJDK 21 (`-XX:+UseContainerSupport` e `-XX:MaxRAMPercentage=75.0`) no Dockerfile, garantindo que o garbage collector dimensione a heap proporcionalmente à quota do container.

Desafio 2: Isolamento de Rede e Portas entre Ambientes (Staging vs Produção)

Problema: Conflito de portas e colisão de dados ao executar instâncias simultâneas de Staging e Produção no mesmo nó de infraestrutura.

Solução: Criação de redes, volumes e nomes de projeto Compose isolados, com Staging nas portas 8081/27018 e Produção nas portas 8080/27017.

Desafio 3: Falhas mascaradas no Smoke Test

Problema: O uso de `|| true` permitia que um endpoint indisponível fosse informado como saudável.

Solução: Uso de `docker compose --wait` e `curl --fail`; qualquer falha agora encerra o job com código diferente de zero.

7. Checklist de Entrega Oficial (100% Atendido)

- ✓ **Projeto compactado em .ZIP com estrutura organizada** — Concluído
- ✓ **Dockerfile funcional** — Concluído (Multi-stage com JDK 21 Alpine e non-root)
- ✓ **docker-compose.yml ou arquivos Kubernetes** — Concluído (Compose Local/Staging/Prod + Manifestos k8s)
- ✓ **Pipeline com etapas de build, teste e deploy** — Concluído (GitHub Actions com 4 jobs)
- ✓ **README.md com instruções e prints** — Concluído (Documentação técnica estruturada)
- ✓ **Documentação técnica com evidências (PDF ou PPT)** — Concluído (Relatório Técnico Oficial)
- ✓ **Deploy realizado nos ambientes staging e produção** — Concluído (ambientes acadêmicos efêmeros e funcionais)